SOFTWARE TESTING

ASSIGNMENT – 2

CO2: TEST CASE DESIGN STRATEGIES

Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator

Student Name	AATHISRI
Register Number	192521252
Course Code	CSA3712
Assignment	Assignment – 2

INTRODUCTION

SwiftShip is an online courier service that calculates parcel shipping charges and tracks parcels until delivery. The service handles more than 40,000 bookings per day. Therefore, even a small calculation error can cause financial loss, incorrect customer billing, complaints and refunds. The purpose of this assignment is to design a reliable and efficient test suite using black-box and white-box testing techniques.

The assignment examines equivalence class partitioning, boundary value analysis, decision-table testing, state-transition testing, control flow graphs, path testing, cyclomatic complexity and coverage. It also evaluates how the suite should change when fuel surcharge and weekend delivery are introduced.

SYSTEM RULES

Rule	Specification
S1 – Weight	Weight must be greater than 0 kg and not more than 50 kg. Weight is accepted up to one decimal place. Otherwise booking is INVALID.
S2 – Zone rate	Zone A = ₹40 per started kg; Zone B = ₹60 per started kg; Zone C = ₹90 per started kg. Any other zone is INVALID.
S3 – Express	Express delivery adds 50% to the charge. Express is allowed only when weight is 30 kg or less. Above 30 kg returns NOT_ALLOWED.
S4 – Discount	Customers with 10 or more parcels booked in the current month receive a 10% discount on the final charge.

Calculation convention: because the charge is applied for every started kilogram, the billable weight is the next whole kilogram for any positive fractional weight. Thus 1.1 kg is charged as 2 kg and 2.0 kg is charged as 2 kg. The formula used in the expected results is: Base charge = ceiling(weight) × zone rate. If express is selected, multiply by 1.5. If monthly parcel count is at least 10, multiply the resulting charge by 0.9.

Important specification observation: S2 says an unknown zone must be INVALID. The supplied pseudocode initializes the rate to ₹40 and changes it only for B or C. Consequently, an unknown zone is incorrectly treated as Zone A. This is a defect that the tests must expose.

TASK 1 – SELECT TEST CASE DESIGN STRATEGIES (5 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Black-box testing designs tests from the external specification without looking at the internal source code. It checks whether the system gives the correct output for a given input. White-box testing uses the program structure, statements, decisions, loops and execution paths to design tests. Both are useful, but they reveal different categories of defects.

System part	Recommended strategy	Information required	Strengths	Limitations
Shipping rules	Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), decision tables	S1–S4 specification	Finds invalid inputs, range-edge defects and interactions among express and discount rules	May miss internal code paths or implementation mistakes not represented in selected input classes
shipCharge pseudocode	Statement, branch, condition and basis-path testing	Source code / pseudocode and control-flow graph	Finds unreachable statements, incorrect decisions, loop defects and missing branches	Cannot prove that the code implements the business requirement correctly
Parcel tracking	State-transition testing	Tracking state model and transition rules	Finds illegal state changes, retry-limit errors and cancellation errors	Does not automatically test charge calculations or all data validation

For the shipping-charge rules, ECP and BVA are the most efficient starting point. ECP divides inputs into groups expected to behave similarly. BVA focuses on values at and immediately around limits such as 0, 30 and 50 kg. A decision table is added because express and discount rules interact.

For the pseudocode, white-box testing is required. The tester maps each decision and loop, then selects tests that execute true and false outcomes and independent paths. For tracking, the state-transition model is the correct source because the validity of an action depends on the parcel's current state.

The strongest approach combines the techniques. Black-box tests check whether the implementation follows the specification, while white-box tests check whether important code structures execute. State-transition tests check the tracking lifecycle. Using only black-box testing could leave internal branches unexecuted; using only white-box testing could execute every branch while still accepting an invalid zone. Therefore, a combined suite gives better defect detection for reasonable effort.

Defects likely to be found: ECP/BVA can find invalid weight acceptance, wrong kilogram rounding and the 30 kg express boundary. Decision tables can find incorrect express/discount combinations. White-box tests can reveal a missing invalid-zone branch and loop behaviour. State tests can reveal cancellation after pickup or more than two delivery retries. Defects unlikely to be found by these tests include infrastructure outages, payment-provider failures and performance problems unless separate non-functional tests are added.

Further evaluation: Test design should begin with risk assessment. The highest risks are incorrect customer charges, accepting invalid parcel details, allowing express delivery above 30 kg and losing control of parcel tracking states. These risks should receive more tests than low-impact display details. A traceability matrix should connect every rule (S1–S4 and each tracking transition) to one or more test cases. This proves that no requirement has been ignored and makes future maintenance easier when the rules change.

TASK 2 – EVALUATE RANDOM TESTING (2 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Random testing generates inputs automatically from selected input ranges. For shipCharge, a generator could select a weight from 0.0 to 50.0, a zone from A/B/C and some invalid zone values, an express flag and a parcel count. Each generated input is passed to the function and the actual result is compared with an independently calculated expected result.

Random testing cannot replace systematic test design. Important values such as 30.0, 30.1 and 50.0 occupy very small portions of a large input space. If weights are generated uniformly from 0.0 to 50.0 in increments of 0.1, there are 501 possible values. The chance of selecting one particular value in one draw is 1/501, approximately 0.20%. In 1,000 independent draws, the probability of hitting a particular value at least once is 1 − (500/501)^1000, approximately 86.5%. This still does not guarantee that the value is tested with the correct zone, express flag and parcel count.

If the generator uses a continuous random number rather than one-decimal values, the probability of hitting exactly 30.0 or 30.1 is effectively zero. The generator must deliberately include boundary values to test them reliably.

A test oracle is the method used to decide the correct output. Random tests are useful only when the expected result can be calculated independently from S1–S4. Reusing the same faulty program logic to calculate expected results would hide defects. Random testing is useful as an additional broad check, but ECP, BVA and decision-table tests must remain the foundation.

A useful improvement is to combine random testing with a fixed seed. A fixed seed makes a failed random test repeatable, so the developer can reproduce the same input and investigate the problem. Random generation should include valid and invalid data, zone values, parcel counts and express combinations. The report should record the random seed, generated input, expected output and actual output. Random testing is valuable as a supplementary technique, but it should not replace carefully selected boundary tests.

TASK 3 – DESIGN AND EVALUATE EQUIVALENCE CLASSES (6 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Equivalence Class Partitioning divides the input domain into groups. One representative value from each group is selected because values in the same class are expected to produce similar behaviour. The following classes are derived directly from S1–S4.

Input	Valid equivalence classes	Invalid / special classes
Weight	0 < weight ≤ 50 kg, up to one decimal place	weight ≤ 0; weight > 50; more than one decimal place (format violation)
Zone	A, B or C	Any other zone value
Express	False; True when weight ≤ 30 kg	True when weight > 30 kg → NOT_ALLOWED
Parcels	0–9 → no discount; 10 or more → 10% discount	Negative or non-integer counts are not defined in the supplied rules; clarify before treating them as invalid

The specification does not explicitly state the permitted data type or lower bound for the parcel count. A practical system should validate that the count is a non-negative integer, but that is an additional assumption and should be confirmed with the product owner.

Representative test cases. The expected charge uses started-kilogram billing.

ID	Weight	Zone	Express	Parcels	Expected result
EC1	1.0	A	No	0	₹40
EC2	1.1	B	No	0	₹120 (2 × 60)
EC3	2.0	C	No	10	₹162 (2 × 90 × 0.9)
EC4	2.0	A	Yes	0	₹120 (2 × 40 × 1.5)
EC5	0.0	A	No	0	INVALID
EC6	50.1	A	No	0	INVALID
EC7	1.0	D	No	0	INVALID
EC8	30.1	A	Yes	0	NOT_ALLOWED
EC9	1.0	A	No	10	₹36 (40 × 0.9)
EC10	1.25	A	No	0	INVALID if one-decimal-place format is enforced

EC1–EC4 represent valid zone, express and discount behaviours. EC5 and EC6 cover the lower and upper invalid weight classes. EC7 checks the invalid-zone class and is expected to fail against the supplied pseudocode because the code returns ₹40 instead of INVALID. EC8 checks the express restriction. EC9 verifies the discount threshold class. EC10 tests the stated precision rule; the pseudocode does not explicitly validate decimal precision.

A single test cannot isolate every invalid class if several invalid inputs are combined. For example, a test with weight 0 and zone D may return INVALID because of weight validation even if zone validation is defective.

Therefore, invalid classes are tested one at a time while the remaining inputs are kept valid. This makes the cause of a failure clear.

The smallest suite depends on whether every class must be independently demonstrated. The ten cases above form a compact representative suite, but a release suite should add at least one additional valid weight and each zone with express/discount combinations. ECP reduces the number of tests; it does not mean that one test proves every possible combination.

The quality of equivalence partitioning depends on whether the classes are mutually exclusive and collectively cover the relevant input domain. Valid weight and invalid weight should not overlap. Zone classes should be separated into A, B, C and all other values. When testing an invalid class, all other fields should be valid so that the test reaches the validation rule under examination. If several invalid fields are supplied together, the first validation error may hide a second defect. This is why one-invalid-condition-at-a-time testing is important.

TASK 4 – DESIGN AND EVALUATE BOUNDARY VALUE TESTS (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Boundary Value Analysis concentrates on values at the edges of input ranges because programming mistakes commonly occur at comparison operators and rounding transitions. The important boundaries are weight 0, the smallest valid weight 0.1, express limit 30.0/30.1, maximum weight 50.0/50.1, started-kilogram transitions and parcel-count threshold 9/10.

Boundary	Test values	Expected behaviour
Minimum weight	0.0, 0.1	0.0 → INVALID; 0.1 → one started kg
Express limit	30.0, 30.1	30.0 with express → allowed; 30.1 with express → NOT_ALLOWED
Maximum weight	50.0, 50.1	50.0 → valid; 50.1 → INVALID
Started kilogram	1.0, 1.1, 1.9, 2.0	1.0 → 1 kg; 1.1 and 1.9 → 2 kg; 2.0 → 2 kg
Discount threshold	9, 10	9 → no discount; 10 → 10% discount

Example expected results using Zone A and no express: weight 0.1 = ₹40; 1.0 = ₹40; 1.1 = ₹80; 1.9 = ₹80; 2.0 = ₹80; 50.0 = ₹2,000. For the express boundary, 30.0 kg in Zone A costs 30 × ₹40 × 1.5 = ₹1,800. At 30.1 kg the result is NOT_ALLOWED. For the discount boundary, 1 kg with 9 parcels costs ₹40, while 1 kg with 10 parcels costs ₹36.

Normal BVA generally tests the nominal value and the values at the minimum and maximum boundaries. Robust BVA also tests values just outside the valid range. Worst-case BVA combines boundary values for multiple variables; it can produce a very large number of tests. For n variables with five boundary choices each, worst-case testing can require 5^n combinations.

For this module, robust BVA is recommended for weight and the express/discount thresholds because invalid values must be rejected safely. A focused combination set is preferable to full worst-case BVA because the module has several input dimensions and the pre-release time is limited. The selected tests should include the exact limits and just-outside values, plus started-kilogram transitions. This offers strong risk coverage without creating an unnecessarily large suite.

Boundary testing should also consider rounding behaviour at integer kilogram transitions. For example, 2.0 kg and 2.1 kg are useful because the billable kilogram count changes from two to three. A defect in the loop or rounding formula may cause the charge to increase too early or too late. Test values immediately below and

above a whole kilogram as well as the overall allowed weight limits. This distinguishes range validation from charge-rounding behaviour.

TASK 5 – APPLY OTHER BLACK-BOX TECHNIQUES (3 Marks)

CO2 | Bloom's Level: K5 – Evaluate

ECP and BVA test individual input classes and boundaries, but they may not detect defects caused by interactions between rules. A decision table is suitable for express and discount because the final amount depends on both conditions.

Condition / Rule	R1	R2	R3	R4
Express selected?	No	No	Yes	Yes
Weight > 30 kg?	—	—	No	Yes
Parcels ≥ 10?	No	Yes	Yes	Either
Express action	No surcharge	No surcharge	Add 50%	Return NOT_ALLOWED
Discount action	No discount	Apply 10%	Apply 10% if allowed	No charge returned
Outcome example	Base	Base × 0.9	Base × 1.5 × 0.9	NOT_ALLOWED

The following tests cover the meaningful combinations. Use 2 kg in Zone A, so the base charge is ₹80.

Test	Express	Weight	Parcels	Expected result
DT1	No	2 kg	9	₹80
DT2	No	2 kg	10	₹72
DT3	Yes	2 kg	9	₹120
DT4	Yes	2 kg	10	₹108
DT5	Yes	30.1 kg	10	NOT_ALLOWED

DT4 is important because it verifies that both the express increase and discount are applied. The expected calculation is ₹80 × 1.5 × 0.9 = ₹108. DT5 verifies that the express restriction takes priority even when the customer qualifies for a discount. ECP/BVA may test express and discount separately but miss the combined calculation or an incorrect order of operations. The decision table therefore adds meaningful interaction coverage.

The decision table acts as a compact requirement model. Each column represents one meaningful rule combination, and each row represents a condition or expected action. Before implementation, the table can be reviewed with the business analyst to confirm the intended order of percentage adjustments. After implementation, the same table becomes a repeatable test basis. This reduces ambiguity about whether a discount is applied before or after an express surcharge.

TASK 6 – DESIGN AND EVALUATE STATE-TRANSITION TESTS (3 Marks)

CO2 | Bloom's Level: K5 – Evaluate

State-transition testing checks whether a parcel moves from one state to another only when the transition is permitted. It is appropriate for tracking because the same action may be valid or invalid depending on the parcel's current state.

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED
↘ FAILED_ATTEMPT
OUT_FOR_DELIVERY → FAILED_ATTEMPT → OUT_FOR_DELIVERY (up to 2 retries)
↘ RETURNED
BOOKED → CANCELLED (only before PICKED_UP)

Current state	Allowed next state(s)	Important invalid transition examples
BOOKED	PICKED_UP, CANCELLED	BOOKED → DELIVERED; cancellation after pickup
PICKED_UP	IN_TRANSIT	PICKED_UP → CANCELLED
IN_TRANSIT	OUT_FOR_DELIVERY	IN_TRANSIT → DELIVERED without delivery attempt
OUT_FOR_DELIVERY	DELIVERED, FAILED_ATTEMPT	OUT_FOR_DELIVERY → CANCELLED
FAILED_ATTEMPT	OUT_FOR_DELIVERY (maximum two retries), RETURNED	Third retry; FAILED_ATTEMPT → DELIVERED directly
DELIVERED	No further transition stated	DELIVERED → IN_TRANSIT or CANCELLED
RETURNED	No further transition stated	RETURNED → OUT_FOR_DELIVERY

Valid transition tests should execute the normal route from BOOKED to DELIVERED, a failed attempt followed by a retry and successful delivery, and a failed attempt sequence that ends in RETURNED after the retry limit. A cancellation test should confirm that BOOKED can be cancelled and that a parcel cannot be cancelled after it reaches PICKED_UP.

The retry wording says a parcel may return to OUT_FOR_DELIVERY a maximum of two retries. The test data should count retries explicitly: first failed attempt → retry 1; second failed attempt → retry 2; after the next failed attempt, the parcel must move to RETURNED and must not be scheduled for another retry. The invalid transitions selected are those likely to cause financial loss, incorrect customer notifications or loss of parcel traceability.

State testing should verify both the current state and the history of transitions. A system might incorrectly allow a third retry if it checks only the current FAILED_ATTEMPT state and does not store the retry count correctly. Tests should inspect the retry counter and resulting state after every failed delivery attempt. The system should also retain an audit record containing the previous state, new state, timestamp and reason for the transition.

TASK 7 – CONSTRUCT THE CONTROL FLOW GRAPH (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

A control flow graph (CFG) represents the order in which statements and decisions may execute. Each node represents one or more sequential statements, and each directed edge represents a possible transfer of control.

Node	Pseudocode line(s)	Meaning
N1	1–2	Validate weight; invalid weight returns INVALID
N2	3–7	Set default rate and select Zone B or Zone C rate
N3	8–9	Initialize charge and working weight
N4	10–12	Loop: add rate and subtract one kilogram
N5	13–16	Check express; reject if weight > 30; otherwise multiply by 1.5
N6	17–18	Check parcel count and apply 10% discount
N7	19	Return charge

START → N1
N1 [weight <= 0 OR weight > 50?] --Yes--> INVALID / END
--No--> N2
N2 [zone == B?; else zone == C?] → N3

N3 → N4 [w > 0?] --Yes--> add rate; w = w - 1 → N4
--No--> N5
N5 [express?] --No----------→ N6
--Yes--> [weight > 30?] --Yes--> NOT_ALLOWED / END
--No--> multiply by 1.5 → N6
N6 [parcels >= 10?] --Yes--> multiply by 0.9 → N7
--No----------→ N7
N7 → END

The line 1 condition is compound: weight <= 0 OR weight > 50. At the decision level it is one if-decision with two atomic conditions. Condition-coverage testing should exercise each atomic condition as both true and false where feasible.

The graph reveals a mismatch between the specification and the code. The rate is initialized to ₹40. The code explicitly checks B and C but has no final else branch to reject other zone values. Therefore, every unrecognized zone is treated as Zone A. The CFG is derived from the code, not from the intended specification, so the missing invalid-zone branch must be recorded as a defect.

The code also does not visibly validate the one-decimal-place precision requirement. If this requirement is enforced by the input interface, it still needs a separate validation test at the interface or service boundary.

The control-flow graph is useful for identifying missing decision logic, but it must be compared with the business specification. The graph represents what the current code does, including its mistakes; it does not automatically represent what the code should do. Maintain two views: the requirement flow, which includes rejection of unknown zones, and the implementation flow, which currently defaults unknown zones to the Zone A rate. Their difference is evidence of a requirement-implementation gap.

TASK 8 – EVALUATE PATH TESTING AND CYCLOMATIC COMPLEXITY (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Cyclomatic complexity estimates the number of linearly independent paths through a program. It helps determine a minimum set of basis paths for decision logic, but it does not prove that all possible loop counts or input combinations have been tested.

Method 1 – Decision count: the pseudocode has seven decision points: weight validation (line 1), zone B check (line 4), zone C check (line 6), loop condition (line 10), express check (line 13), express weight check (line 14), and discount check (line 17). Therefore V(G) = decisions + 1 = 7 + 1 = 8.

Method 2 – CFG formula: V(G) = E − N + 2P, where E is the number of edges, N is the number of nodes and P is the number of connected components. For one connected control-flow graph, P = 1. A node/edge count using the graph above gives the same result, V(G) = 8. The compound OR at line 1 is counted as one decision node for this CFG calculation; condition coverage separately examines its two atomic conditions.

A representative basis-path set is shown below. Each path changes at least one decision outcome relative to the other paths. Inputs are chosen to make the paths feasible.

Path	Path focus	Example test data	Expected result / purpose
P1	Invalid weight branch	weight=0, zone=A, express=false, parcels=0	INVALID
P2	Zone B branch; loop; no express; no discount	1 kg, B, false, 0	₹60
P3	Zone C branch; no express; discount	1 kg, C, false, 10	₹81
P4	Zone A default; express false; discount false	1 kg, A, false, 0	₹40

P5	Express true; weight > 30	30.1 kg, A, true, 0	NOT_ALLOWED
P6	Express true; weight ≤ 30	2 kg, A, true, 0	₹120
P7	Discount true after express	2 kg, A, true, 10	₹108
P8	Loop repetition / multiple iterations	3 kg, A, false, 0	₹120; loop executes three times

The exact independent path set depends on the final CFG node grouping. The table is a practical basis for covering the eight decision-derived paths and important outcomes; some paths overlap in their executed statements. The zero-iteration loop path is infeasible for a valid weight because S1 requires weight > 0 and the loop starts with w = weight. A valid input therefore causes at least one loop iteration. Test one iteration (1 kg) and multiple iterations (for example, 3 kg) to check loop entry and repetition. Also test a fractional weight such as 1.1 kg to verify that the loop performs two iterations.

Path testing is not sufficient by itself. The number of possible paths grows rapidly when loops repeat, and the loop can execute many times. It is also possible to execute a path while using a value that exposes an incorrect business rule. Therefore, basis paths must be combined with specification-based tests.

A basis path is an independent route through the control-flow graph that introduces at least one new edge not covered by previously selected basis paths. Cyclomatic complexity gives the theoretical minimum number of independent paths for the graph, not the total number of tests needed for all input values. A loop creates many possible execution counts, so testers select representative loop tests. In this program, zero loop iterations cannot occur for a valid positive weight, because the loop starts with w = weight. One and multiple iterations must still be tested.

TASK 9 – JUDGE TEST ADEQUACY AND COVERAGE (3 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Coverage measures how much of a selected testing target has been exercised. No single percentage proves that a program is defect-free. For a money calculation used by more than 40,000 parcels per day, branch coverage should be treated as a minimum, with condition coverage and specification-based tests added for high-risk rules.

Coverage criterion	Meaning for shipCharge	Value and limitation
Statement coverage	Every executable statement runs at least once	Can miss the false outcome of a decision
Branch coverage	Every decision outcome (true and false) runs	Stronger than statement coverage; still may miss individual atomic condition effects
Condition coverage	Each atomic condition is true and false at least once	Useful for weight OR condition and other compound logic
Multiple-condition coverage	All combinations of atomic conditions in a compound decision are exercised	Stronger, but may increase test count; infeasible combinations must be considered

The path tests in Task 8 execute the invalid and valid weight outcomes, B and C zone selections, loop entry and repetition, express true/false, express rejection/acceptance, and discount true/false. To make the weight compound condition's condition coverage explicit, use weight = 0 (first condition true, second false) and weight = 50.1 (first false, second true), plus a valid value such as 10 (both false). This ensures each atomic condition is independently true and false.

To reach stronger coverage, add tests for the individual condition combinations and all relevant rule combinations, including invalid zone values. 100% statement or branch coverage would not necessarily find

the invalid-zone defect if tests only use zones A, B and C. The defect is a missing requirement check, not merely an unexecuted existing branch. A test with zone D is essential.

Recommendation: require 100% statement and branch coverage for this small calculation module, condition coverage for compound decisions, and explicit ECP/BVA/decision-table tests for business rules. Record any infeasible condition combinations. Coverage should be used with defect-based evidence, not as a standalone release decision.

Coverage reports should identify the exact denominator and the tests that contributed to the result. Branch coverage is the number of executed decision outcomes divided by the total possible decision outcomes. Statement coverage can look high even when an important false branch is never executed. Coverage tools also cannot identify every missing requirement because a requirement with no corresponding code branch may not appear in the report. This explains why an invalid-zone test remains necessary even when structural coverage is complete.

TASK 10 – COMPARE BLACK-BOX AND WHITE-BOX EFFECTIVENESS (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Black-box tests (Tasks 3–6) are derived from the business specification. White-box tests (Tasks 7–9) are derived from the program structure. Their effectiveness should be judged by the defects they expose, not simply by the number of test cases.

Defect / risk	Black-box contribution	White-box contribution
Unknown zone accepted as Zone A	Directly finds it using zone D or another invalid value	May reveal missing validation branch when CFG is compared with specification, but code coverage alone may not find it
Wrong started-kilogram calculation	Finds through 1.1 kg and similar boundary tests	Finds loop iteration errors with one and multiple iterations
Express limit comparison error	Finds through 30.0 and 30.1 kg tests	Exercises both outcomes of line 14
Discount threshold error	Finds through 9 and 10 parcel tests	Exercises both outcomes of line 17
Incorrect express/discount interaction	Finds through decision-table combinations	Can execute both decisions, but needs expected-result checking
Illegal tracking transition	Finds through state-transition tests	Not covered by shipCharge CFG because tracking is a separate module

Black-box testing is especially effective at detecting incorrect business behaviour, such as the invalid-zone defect and wrong charges. White-box testing is effective at finding unexecuted statements, missed branches and loop-related implementation problems. Some defects are found by both, such as an incorrect express comparison when a boundary test is also used to execute the relevant branch.

Overlapping tests can be consolidated if they use the same inputs and verify the same expected result while preserving the required coverage. For example, a 30.1 kg express test can satisfy both BVA and branch coverage for the NOT_ALLOWED branch. However, the test should retain both labels in the test documentation so the evidence is clear.

The result may differ for other modules. A user-interface module may benefit more from black-box usability and workflow tests, while a complex algorithm may need stronger white-box path and data-flow testing. The best mix depends on the module's risk, complexity and specification quality.

A fair comparison should use the same defect list and execution results for both approaches. Defects found only by black-box testing show that specification-based input selection added value beyond code coverage. Defects found only by white-box testing show that internal structures were not adequately represented by the

external test set. Defects found by both approaches show useful overlap. The goal is not to remove all overlap, because a critical financial rule may deserve independent confirmation.

TASK 11 – RECOMMEND AND DEFEND THE FINAL TEST SUITE (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

The final regression suite should be small enough to run before every release but broad enough to protect the high-risk rules. The following compact suite combines ECP, robust BVA, decision-table testing, white-box path coverage and state-transition tests. Repeated cases can serve more than one coverage objective.

ID	Test purpose / technique	Input or action	Expected result
R01	Minimum valid weight – BVA	0.1 kg, A, no express, 0 parcels	₹40
R02	Invalid lower weight – robust BVA	0 kg, A, no express, 0	INVALID
R03	Maximum valid weight – BVA	50 kg, A, no express, 0	₹2,000
R04	Above maximum – robust BVA	50.1 kg, A, no express, 0	INVALID
R05	Started kg – BVA	1.1 kg, A, no express, 0	₹80
R06	Zone B – ECP / path	1 kg, B, no express, 0	₹60
R07	Zone C + discount – ECP	1 kg, C, no express, 10	₹81
R08	Invalid zone – ECP	1 kg, D, no express, 0	INVALID (current code defect)
R09	Express limit – BVA	30 kg, A, express, 0	₹1,800
R10	Express over limit – BVA	30.1 kg, A, express, 0	NOT_ALLOWED
R11	Discount boundary – BVA	1 kg, A, no express, 9 parcels	₹40
R12	Discount boundary – BVA	1 kg, A, no express, 10 parcels	₹36
R13	Express + discount – decision table	2 kg, A, express, 10 parcels	₹108
R14	Loop multiple iterations – white-box	3 kg, A, no express, 0	₹120
R15	Normal tracking route – state transition	BOOKED through DELIVERED	All valid transitions accepted
R16	Cancellation restriction – state transition	Attempt cancellation after PICKED_UP	Cancellation rejected
R17	Retry limit – state transition	Two retries used; next failed attempt	Move to RETURNED; no third retry

This is a risk-based regression suite rather than an exhaustive combination of every input. It checks the minimum and maximum weight, started-kilogram rounding, all valid zones, invalid zone, express boundary, discount threshold, interaction of express and discount, loop repetition and important tracking rules. The invalid-zone test should fail on the supplied pseudocode until the implementation is corrected.

Before each release, automate the calculation tests where possible and run the tracking tests through the relevant service or test environment. Store the input, expected result, actual result, pass/fail status and defect reference. The suite should be measured against the 30-minute execution target introduced in Task 12. Known remaining gaps include performance under peak booking volume, payment integration, network failures, date/month rollover for parcel counts and security. These require separate non-functional and integration tests.

Each regression test should have a stable identifier, clear preconditions, input values, expected result, actual result, pass/fail status and linked requirement. Test data should be reset between cases so that one customer's parcel count or tracking history does not affect another case. The suite should distinguish automated calculation tests from end-to-end tracking tests. This makes execution time easier to estimate and flaky tests easier to identify.

TASK 12 – RE-EVALUATE AND ADAPT THE TEST SUITE (8 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Changed requirement

SwiftShip introduces two new features: an 8% fuel surcharge on Zone C parcels and a weekend delivery option that adds ₹50 per parcel. Developers add two new decisions after line 16. The regression suite must continue to run within 30 minutes.

A. Impact Analysis (2 Marks)

The specification now has two additional rules. First, if the parcel's zone is C, an 8% fuel surcharge is applied. Second, if weekend delivery is selected, ₹50 is added per parcel. The input model therefore gains a weekend flag, and the calculation logic gains a Zone C surcharge decision and a weekend-delivery decision.

The affected equivalence classes include the zone classes because Zone C now has an additional calculation; the weekend option has two classes, selected and not selected. The weight, express and discount boundaries remain important. The decision table must be expanded to include Zone C versus Zones A/B and weekend selected versus not selected.

The control flow graph gains two decision nodes if both new requirements are implemented as separate if-statements. Cyclomatic complexity therefore increases by two, from 8 to 10, assuming no additional compound conditions or branches are introduced. The new tests must execute true and false outcomes of both decisions.

The 30-minute limit matters because the regression suite must be repeatable before every release. An extremely large worst-case combination suite may exceed the limit and delay releases. Risk-based selection, test automation, reusable setup and prioritizing high-impact rules help maintain execution within the time limit.

B. Evaluation of the Original Suite (2 Marks)

Most original tests remain useful. Weight boundaries, started-kilogram tests, express tests, discount tests, invalid-zone validation and tracking tests are still valid. However, expected results for Zone C tests must be recalculated to include the new 8% fuel surcharge. The original suite does not test weekend delivery because that input did not previously exist.

New paths not covered by the original suite include the Zone C surcharge being applied and not applied, weekend delivery selected and not selected, and combinations of the new surcharge with express, discount and weekend fee. The original coverage target is no longer enough because two new decisions have been added. The test suite must be extended to cover both outcomes of each new decision.

For clarity, the specification does not state whether the fuel surcharge is applied before or after express and discount, nor whether the weekend fee is discounted or increased by express. The phrase 'adds ₹50 per parcel' suggests a fixed addition, but the exact order of operations should be confirmed. The expected results below use the explicit assumption that the Zone C fuel surcharge is 8% of the base shipping charge and the weekend fee is added as a fixed amount after percentage calculations. This assumption must be confirmed with the product owner before release.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original suite. A complete redesign is unnecessary because most original tests still represent important rules. The suite should retain the stable tests, update Zone C expected values and add tests for the new decisions and their interactions.

New / updated test	Input	Expected result under stated assumption
M01 – Zone C fuel surcharge	1 kg, C, no express, 0 parcels, weekday	₹97.20 (₹90 + 8% of ₹90)

M02 – Non-Zone C no fuel surcharge	1 kg, A, no express, 0 parcels, weekday	₹40
M03 – Weekend fee	1 kg, A, no express, 0 parcels, weekend, 1 parcel	₹90 (₹40 + ₹50)
M04 – Weekend not selected	1 kg, A, no express, 0 parcels, weekday	₹40
M05 – Zone C + weekend	1 kg, C, no express, 0 parcels, weekend, 1 parcel	₹147.20 (₹90 + ₹7.20 + ₹50)
M06 – Zone C + discount	1 kg, C, no express, 10 parcels, weekday	₹87.48 if surcharge is applied before 10% discount: ₹90 × 1.08 × 0.9
M07 – Zone C + express	2 kg, C, express, 0 parcels, weekday	₹291.60 if surcharge is applied to base before express: ₹180 × 1.08 × 1.5

The suite's coverage increases by adding tests for the two new decision outcomes and selected combinations. The old R07 Zone C test must be updated. M01 and M02 exercise the Zone C surcharge decision's true and false outcomes; M03 and M04 exercise weekend selection and non-selection. M05–M07 check interactions with weekend, discount and express. The invalid-zone test remains essential and should be fixed in the code.

To meet the 30-minute limit, keep a compact automated regression set for calculation logic and run state-transition tests in parallel where the environment permits. Avoid generating every possible combination. Use the decision table and risk assessment to select combinations that can change the amount charged.

D. Evidence-Based Defence (2 Marks)

The modified suite is supported by five types of evidence. ECP covers valid and invalid weight, zone, express, parcel-count and weekend classes. BVA covers 0/0.1 kg, 30/30.1 kg, 50/50.1 kg, started-kilogram transitions and 9/10 parcels. Decision-table testing covers express and discount interactions as well as Zone C surcharge and weekend delivery. White-box testing covers the updated control-flow decisions and independent paths. State-transition tests protect the tracking lifecycle and retry/cancellation restrictions.

Cyclomatic complexity is expected to rise from 8 to 10 because two decisions are added. The basis-path set must therefore include paths that exercise both outcomes of the new decisions. The suite should report statement, branch and condition coverage for the revised function, together with explicit specification tests for invalid zone and new charge combinations.

The final recommendation is to modify and automate the original suite, not retain it unchanged or redesign it entirely. This preserves previously valuable tests, adds the missing new behaviour and controls execution effort. A timed run must confirm that the final regression suite completes within 30 minutes. If it exceeds the limit, prioritize high-risk calculation and transition tests for every release and schedule the remaining extended combinations separately.

Change-impact analysis should be completed before editing the regression suite. Identify every function, decision, interface and stored data field affected by the fuel surcharge and weekend option. Review unclear calculation order, rounding rules and whether the weekend amount is charged per parcel or per booking. After clarification, update expected results, add tests for both outcomes of the new decisions, rerun regression tests and compare coverage and execution time with the previous release.

CONCLUSION

A reliable SwiftShip test suite requires both specification-based and code-based testing. Equivalence classes and boundary values efficiently test input ranges, decision tables expose interactions, state-transition tests protect parcel tracking, and control-flow and path testing examine the implementation. Coverage metrics provide useful evidence but cannot replace carefully selected tests derived from business rules.

The supplied pseudocode contains an important invalid-zone defect because unknown zones receive the default Zone A rate. The new fuel surcharge and weekend option also require updates to expected results, decision tables, control-flow analysis and regression coverage. Therefore, the best decision is to modify the original suite, add targeted tests and automate the repeatable checks so that the final suite remains effective within the 30-minute release limit.

Practical execution and reporting: Before execution, confirm the build version, test environment, valid test accounts and clean parcel data. Execute high-priority financial tests first, followed by tracking lifecycle tests and extended combinations. Log each failure with a reproducible input, expected and actual result, severity and linked requirement. After a fix, rerun the failed test and related regression cases to ensure that the correction has not introduced another defect.

REFERENCES

- Software Testing – Assignment 2, CO2: Test Case Design Strategies, SwiftShip scenario and specification supplied in the assignment question.
- ISTQB Foundation Level concepts: equivalence partitioning, boundary value analysis, decision-table testing, state-transition testing and structural coverage.
- Standard software testing concepts: control-flow graphs, cyclomatic complexity, basis-path testing and regression testing.